

Reasoning Under Projection: Structural Failure Modes and the Admissibility Framework

Brian Cameron

June 19, 2026

Abstract

AI systems increasingly make recommendations, explanations, security judgments, and agentic decisions from compressed representations: prompts, summaries, retrieved snippets, logs, metrics, traces, policy artifacts, and interface-bounded context. These representations often omit or collapse distinctions that remain operationally relevant. The relevant failure is a failure of invariant preservation: the dependency structure required by a downstream claim no longer survives projection in a form that supports the requested answer. When a downstream claim depends on a distinction that the representation does not preserve or make boundedly recoverable, the failure is not merely uncertainty or correctable error. It is representational loss.

This paper identifies a structural failure mode in AI reasoning systems: non-identifiability under projection is misclassified as recoverable inference error. In such cases, additional reasoning over the same representation can make an answer more fluent, coherent, or persuasive without restoring the missing distinction. This produces familiar safety and security failures: unsupported root-cause attribution, hallucinated source support, unsafe action under missing state, and metric-driven proxy success, a failure mode related to Goodhart-style proxy collapse [5].

The paper distinguishes error, which is correctable within the current representation, from loss, which requires changing the information basis or answer form. It then introduces a dependency-status ladder organized by the answer forms licensed after projection: preserved, reconstructible, traceable, or opaque. Finally, it describes structural requirements for reliable reasoning under compression: observability of loss-relevant state, bounded correction, and non-collapse of distinct representational levels.

The goal is not to eliminate uncertainty or require abstention. It is to ensure that reasoning systems do not present conclusions as determined by a representation when the representation does not support them. The resulting admissibility framework treats response form as part of structural reliability: a system should answer directly when support is preserved, reconstruct when bounded recovery is available, mark or route traceable dependencies, and refuse or reformulate claims whose dependencies are opaque. This paper develops the conceptual framework and identifies several directions for future formalization, operationalization, and evaluation.

1 Introduction: Local Coherence Without Structural Admissibility

This paper is intended as a conceptual bridge between formal accounts of identifiability and practical concerns in AI reasoning, safety, security, and agentic reliability. Its central claim is that many AI reasoning failures are not simply wrong inferences, but unsupported inferences from representations that do not preserve the distinctions required by the query. This matters for systems that answer from prompts, summaries, retrieval results, logs, traces, metrics, policy artifacts, or compressed

context windows: the system may produce a locally coherent answer while the representation itself does not support the conclusion.

As a running example, consider an AI assistant used during a security incident. It receives a compressed summary of logs, deployment history, alerts, and recent configuration changes. The assistant produces a clear root-cause explanation and recommends a fix. The reasoning may appear locally coherent: it cites the deployment, explains the failure, and proposes remediation. However, the compressed representation may not preserve the distinction between a deploy-triggered bug, a dependency outage, a configuration interaction, and telemetry loss caused by reduced logging. If the available representation cannot distinguish these states, then the assistant has not merely made an error. It has produced a determinate explanation from a representation that does not support one.

More generally, the problem can be understood as one of invariant preservation under projection. A claim remains admissible only when the distinctions on which it depends are preserved, boundedly reconstructible, or explicitly marked as unavailable. Projection is dangerous not because it compresses, but because it can collapse the invariants required by a downstream claim while leaving the answer form unchanged. Good reasoning under projection therefore requires preserving the dependency structure needed by the claim, or changing the response form when that structure no longer survives.

The novelty claimed here is not the invention of identifiability, abstraction, partial observability, sufficiency, or information preservation. The contribution is to identify a reasoning-system failure mode that occurs when a system behaves as though the dependency status required by a query survived projection when it did not. Existing frameworks can describe hidden state, abstraction, sufficiency, or non-identifiability; the present framework asks how a reasoning system should regulate answer form when the representation available to it no longer supports the distinction the answer requires.

In AI safety and security settings, the dangerous failure is often not that a model is uncertain. It is that the model behaves as though a missing distinction were recoverable from the representation available to it. This can occur when a security assistant recommends a fix without the relevant code path or threat model, when a RAG system treats a citation pointer as source support, when an incident-response assistant names a root cause despite indistinguishable telemetry, or when an agent acts as if authorization, intent, or environmental state were known from compressed memory. These issues connect to broader concerns about robustness and failure modes in AI systems [1].

These failures are structurally similar. The system is not merely making a bad guess; it is presenting a claim with a stronger dependency status than the representation supports. The missing distinction may be traceable, ambiguous, externally recoverable, or opaque, but it is not preserved in the current representation. Treating such loss as ordinary error encourages overconfident answers, hallucinated support, unsafe action, and misleading explanations.

The account is normative in a limited epistemic sense. It does not describe how reasoning systems always behave or prescribe substantive decisions or values. It proposes admissibility criteria for reasoning under compression: a system should not present a conclusion as determined by a representation when the relevant query depends on a distinction that the representation does not preserve, cannot boundedly reconstruct, and cannot justify without changing the information basis or answer form. Admissibility is therefore query-relative. A projection is not defective merely because it compresses; it becomes inadmissible for a particular use when the downstream claim depends on a collapsed distinction.

The resulting evaluation question is therefore directly relevant to AI reasoning, safety, and security evaluation. It asks whether a system can distinguish supported inference from decision under non-identifiability; whether it can mark when source support is traceable but not reconstructible;

whether it can avoid collapsing system-level failures into single actors; and whether it can keep correction bounded to the information actually available. These conditions matter because safety failures often occur at the boundary between what the system can infer, what it must assume, and what it should route, defer, or refuse.

This paper makes three contributions. First, it characterizes a recurring structural failure mode: unsupported determinacy under projection, where a system presents a determinate answer even though the operative representation does not preserve the distinctions required by the claim. Second, it distinguishes recoverable error from representational loss using identifiability under projection. Third, it introduces a dependency-status ladder—preserved, reconstructible, traceable, and opaque—that links structural support to admissible response forms. This paper is conceptual rather than complete: it identifies the structural failure mode, defines the dependency-status vocabulary, and outlines directions for future formalization, operationalization, evaluation, and control.

2 Reasoning Under Projection

To understand these failures, we must examine the conditions under which reasoning operates. In practice, reasoning systems do not operate on complete representations of the problems they address.

2.1 Reasoning Over Representations

Any reasoning process operates on a representation of the problem rather than the full underlying system. In AI systems, that representation may be a prompt, retrieved context, tool output, memory state, log summary, embedding, schema, score, or policy artifact. The question is not whether the representation is complete; it is whether it preserves the distinctions required by the claim being made.

Operative representation in LLM systems. For language models, the operative representation is not simply the user prompt. It includes the information basis available to the model for the current answer: the prompt, conversation state, retrieved passages, tool outputs, exposed memory, structured metadata, policy constraints, explicit assumptions, uncertainty markers, and any prior or background knowledge that the system is permitted to rely on for the task. This boundary is often fuzzy. The framework does not require that all internal model state be inspectable. It requires that claims be calibrated to the support that is available, exposed, or explicitly invoked in the answer. If a claim depends on latent background knowledge, an unavailable source, a hidden prior, or a tool result not present in the answer context, then that dependency should be marked rather than presented as if it were determined by the visible representation alone.

2.2 Projection as Compression

We define *projection* as any transformation that maps a richer system state into a reduced representation used for reasoning. Projection may be lossy in the information-theoretic sense [10], but the relevant issue for reasoning is whether the reduced representation preserves the distinctions required for later inference, decision, or evaluation.

Formally, projection can be understood as a mapping:

$$P : S \rightarrow R$$

where S is the full system state and R is the representation used for reasoning.

Multiple distinct states in S may map to the same representation in R .

A simple example illustrates the query-relative nature of projection. A black-and-white image may preserve shape while losing color. It remains admissible for many shape queries, but not for a query that depends on whether an object was red or green. The projection is not defective in general; it is insufficient for queries whose answers depend on the collapsed distinction.

In this paper, projection refers specifically to transformations that induce non-trivial equivalence classes over the underlying state space: distinct states in S are mapped to the same representation in R .

The analysis is relative to a given projection P . A query may fail to be identifiable under one representation while becoming identifiable under a richer or differently structured representation. For example, a log summary may preserve that an outage occurred while losing the event ordering needed for root-cause analysis. A retrieval result may preserve a citation identifier while omitting the source passage needed to verify the claim. A risk score may preserve a ranking while losing the distinction between reversible and irreversible failures. These projections are useful, but they are not equally admissible for every downstream query.

In practice, however, reasoning systems often operate under fixed or externally imposed projections: a prompt, summary, metric, ranking, category, analogy, translation, dataset schema, interface boundary, token budget, or institutional decision format may determine what information is available at inference time. The question addressed here is therefore not whether some better representation could exist, but what follows when reasoning must proceed under the representation actually available.

2.3 Equivalence Classes and Collapse of Distinction

Because projection is many-to-one, it induces equivalence classes over the original state space. Distinct underlying states become indistinguishable once projected into the representation.

This means that some distinctions present in the original system may no longer be available to the reasoning process through the current representation. For the query being asked, those distinctions are not merely hidden within the representation; they are unavailable unless additional assumptions, structure, or observations are introduced.

This is the point at which ordinary partial observability and representational loss diverge. In a partial-observation setting, the system may maintain a belief state over hidden possibilities still represented by the model. Under projection-induced loss, the distinction required by the query may not be represented in the current state space at all. No update over the current representation can recover such a distinction unless the representation is expanded, a bounded reconstruction path is available, or an explicit assumption is introduced and marked.

2.4 Identifiability Under Projection

Let $P : S \rightarrow R$ be a projection from system states to representations. The projection induces an equivalence relation on S :

$$s_1 \sim_P s_2 \quad \text{iff} \quad P(s_1) = P(s_2).$$

A query, judgment, or decision-relevant property $q : S \rightarrow A$ is identifiable under P when it is constant over the equivalence classes induced by P :

$$P(s_1) = P(s_2) \Rightarrow q(s_1) = q(s_2).$$

The use of identifiability here generalizes the term beyond parameter recovery in statistical or causal models. In this paper, identifiability refers to whether any reasoning-relevant query, judgment, or decision property survives projection into the representation used by the system.

If this condition holds, then the representation preserves enough information to answer q . If the condition fails, then two states that are indistinguishable in the representation require different answers to q . In that case, no reasoning process operating only on R can determine q without introducing additional assumptions, expanding the representation, or obtaining new information.

This gives a compact way to state the error–loss distinction. An error occurs when q is identifiable under the projection but the reasoning process returns the wrong value. Loss occurs when q is not identifiable under the projection, so the representation itself no longer supports a determinate answer.

Minimal example. Let $S = \{s_1, s_2\}$ and let $P(s_1) = P(s_2) = r$. Let $q(s_1) = 0$ and $q(s_2) = 1$. Then q is not identifiable under P , because the same representation r requires two different answers. No function $f : R \rightarrow A$ can recover q , since $f(r)$ must return a single value. If instead $q(s_1) = q(s_2)$, then q is identifiable under P even though P collapses the two states.

Structural Observation 1: Non-identifiability blocks recovery from the projected representation alone. If a query $q : S \rightarrow A$ is not identifiable under projection $P : S \rightarrow R$, then no function $f : R \rightarrow A$ can recover q for all states in S .

Argument. If q is not identifiable, then there exist $s_1, s_2 \in S$ such that $P(s_1) = P(s_2) = r$ but $q(s_1) \neq q(s_2)$. Any function f over R must assign a single value to r , so it cannot equal both $q(s_1)$ and $q(s_2)$. Therefore, any determinate answer requires an additional assumption, representation expansion, or information outside R .

The formal claim is limited: non-identifiability means that no function over the current representation can recover the query for all underlying states. The design question is what a reasoning system should do when this condition holds. The remaining sections describe structural requirements and behavioral consequences, rather than additional formal impossibility results.

This separates the descriptive and normative parts of the argument. The descriptive claim is that non-identifiability prevents recovery of q from R alone. The normative claim is an epistemic admissibility criterion: a reasoning system should not present a conclusion as determined by R when the conclusion depends on distinctions that R does not preserve or make reconstructible. This criterion does not require inaction. It requires that action under non-identifiability be marked as a decision rule, assumption, conditional response, request for representation expansion, or unresolved dependency rather than as an inference licensed by the current representation. Non-identifiability does not imply silence; it changes the epistemic status of speech. For brevity, I refer to these moves—representation expansion, external information, explicit assumptions, conditionalization, routing, or refusal of the requested form—as changing the information basis or answer form.

Admissibility is intentionally weaker than correctness. The framework does not determine which decision procedure a system must use when R is insufficient, nor does it prohibit probabilities, confidence intervals, ranked hypotheses, priors, utility functions, or policy choices. It constrains the epistemic status of the output: a claim should not be presented as following from R alone when it depends on distinctions that R does not preserve or make reconstructible. A probabilistic or decision-theoretic response may be admissible if it marks the assumptions, priors, utilities, or decision rules on which it depends; it becomes inadmissible when those elements are presented as if they were determined by the representation itself. This is especially important for AI safety and security because many unsafe outputs are not unsupported in the sense of being nonsensical; they

are unsupported in the narrower sense that they present assumptions, priors, tool traces, or policy choices as if they were determined by the available evidence.

Immediate consequences. This observation yields three immediate consequences for reasoning under projection.

Unsupported determinacy. If q is not identifiable under P , then any determinate answer to q from R alone must depend on assumptions not contained in R .

Correction boundary. If a failure concerns an identifiable query, correction can in principle operate within the current representation. If the query is not identifiable, correction requires changing the information basis or answer form; otherwise it is not correction but hidden reconstruction.

Decision/inference separation. A system may still act when q is non-identifiable, but the action must be marked as a decision rule under non-identifiability rather than as an inference determined by R .

Operational corollaries. The observation also yields several testable consequences for reasoning systems.

Representation-enrichment monotonicity. If a representation R' refines R by distinguishing states that R collapses, then some queries non-identifiable under R may become identifiable under R' . A projection-aware system should therefore become more willing to make determinate claims when the enriched representation restores the distinction required by the query, while avoiding such claims under the compressed representation.

Traceability is not reconstruction. Adding a provenance pointer, citation identifier, tool-call record, or source hash to R may make a dependency traceable, but it does not make the query identifiable unless the added representation also preserves or recovers the query-relevant distinction. A system should therefore treat pointer-only support differently from source-content support.

Answer-form sensitivity. When q is non-identifiable under R , a direct determinate answer and a conditional answer do not have the same admissibility status. The direct answer presents q as determined by R ; the conditional answer marks the additional assumption or decision rule on which the claim depends. The framework therefore predicts that evaluation should score not only content, but whether the answer form matches dependency status.

Diagnostic contrast. A benchmark can test the distinction between error and loss by comparing paired tasks that differ only in whether the query-relevant distinction is represented. If a model gives the same determinate answer form in both the compressed and enriched condition, then it is failing to regulate claims by dependency status. If it withholds, branches, or marks assumptions in the compressed condition and becomes determinate only in the enriched condition, then it is behaving in a projection-sensitive way.

2.5 When Projection Produces Representational Loss

Projection produces representational loss for a query when it collapses a distinction required by that query and the distinction is not otherwise encoded, boundedly reconstructible, or explicitly supplied by a marked assumption. The same projection may remain admissible for other queries whose answers are constant across the collapsed states. Thus loss is query-relative: the representation fails not because it is compressed, but because a downstream claim depends on a distinction the compression does not preserve.

The practical implication is that a reasoning system must either change the information basis or change the answer form. It may request additional information, retrieve a source, expose an assumption, branch over alternatives, weaken the claim, route the query, or refuse the requested

form. What it must not do is present a determinate answer as if the missing distinction had been recovered from the original representation.

3 Classifying Failure: Error vs Representational Loss

The failures described in the previous sections arise from a single structural confusion: the treatment of representational loss as if it were recoverable error. Distinguishing between these two cases is necessary for any reasoning process that operates over projected representations.

Using the notation above, error concerns failures in answering an identifiable query; loss concerns cases where the query is not identifiable from the current representation. The distinction matters because treating the second case as the first leads systems to repair, infer, or explain beyond what the representation supports.

3.1 Error: Recoverable Deviation Within a Representation

An *error* occurs when a system produces an incorrect result despite the necessary information being present in the representation. In this case, the failure is due to the reasoning process rather than the representation itself.

Errors are, in principle, correctable. Given the same representation, a different or improved reasoning procedure can recover the correct result. Correction does not require access to information outside the representation; it requires only a revision of how the existing information is used.

Typical examples include:

- arithmetic or logical mistakes,
- incorrect application of rules,
- inconsistent intermediate steps.

In each case, the distinction between correct and incorrect outcomes exists within the representation, and a valid correction can be constructed without introducing new information.

3.2 Loss: Non-Identifiability Under the Current Representation

A *loss* occurs when the representation no longer contains, or cannot validly reconstruct, the information required to distinguish between multiple operationally relevant underlying states. In this case, no reasoning process operating solely on the current representation can recover the missing distinction without additional assumptions, representation expansion, or external information.

The term “loss” is used here as a query-relative category rather than as a claim that all missing structure has the same cause. A query may become non-identifiable because relevant variables are absent, because multiple causal models remain observationally equivalent, because a metric collapses distinct evaluative dimensions, because provenance is unavailable, or because source-level support is not represented. These mechanisms differ, but they share the same operational consequence: the current representation does not preserve enough structure to determine the requested query without additional assumptions, representation expansion, or external information.

Loss names a structural condition of projection rather than an inference failure: once distinct states have been mapped to the same representation, they are indistinguishable from the perspective of the reasoning system.

Typical examples include:

- multiple causal models that produce identical observable outcomes [8],
- decisions that compress competing values into a single metric [9],
- incomplete information where relevant variables are not represented.

In these cases, there is no correct answer that can be derived from the representation alone. Any attempt to produce a single determinate conclusion requires introducing assumptions that are not justified by the available information.

3.3 Misclassification: Treating Loss as Error

Many reasoning failures occur when a system treats loss as if it were error. That is, the system behaves as though additional reasoning steps can recover distinctions that are no longer present in the representation.

This misclassification leads to several characteristic behaviors:

- **Overconfidence:** The system presents a single conclusion without acknowledging indistinguishability, such as naming a root cause when telemetry cannot distinguish among plausible causes.
- **Hallucination:** Missing information is implicitly or explicitly fabricated, such as citing source support that is not present in the retrieved context.
- **Arbitrary resolution:** One of several equivalent possibilities is selected without justification, such as choosing a security fix before the vulnerable code path is represented.
- **Unsafe action:** An agent acts as if authorization, environmental state, user intent, or reversibility is known when those distinctions are not preserved in its current state.

In each case, the system produces an output that appears well-formed, but the reasoning process is no longer grounded in recoverable state.

3.4 Detection and Representation of Loss

Unlike error, loss cannot be corrected through additional reasoning over the same representation alone. The system must instead change the epistemic status of the claim or change the representation on which the claim depends. It may request additional information, expand or refine the representation, reconstruct the distinction through an explicit bounded path, weaken the claim into a conditional, or mark and trace the loss. Marking and tracing preserve accountability for the missing distinction, but they do not make claims depending on that distinction admissible.

Tracing does not recover the missing distinction. It records the dependency, assumption, or point of loss so that downstream claims are not treated as if the distinction were preserved. The response may therefore take the form of ambiguity marking, branching over multiple consistent explanations, separating inference from decision, or routing the query to a source or procedure that can restore the missing distinction.

In some cases, representation expansion is not simply a matter of adding more facts to the current context. The missing distinction may be available only through an external source, measurement procedure, institutional role, policy, or decision-maker. In such cases, an admissible response may route the query to the appropriate source, procedure, role, or authority rather than answer from the current representation. The relevant point is not social authority as such, but

dependency status: the current representation does not contain the distinction, and the system must identify where that distinction could be preserved, reconstructed, or validated.

The key requirement is that the system does not collapse indistinguishable states into a single unjustified conclusion.

4 The Dependency-Status Ladder

For a query q and representation R , *dependency status* refers to the kind of support that R provides for the distinctions on which an answer to q depends. It is not identical to provenance, confidence, observability, or identifiability, although it may involve each of them. Identifiability asks whether a function from R to the answer exists. Dependency status asks what kind of support the current reasoning process has for using that answer: whether the needed distinction is directly preserved, recoverable through a bounded procedure, merely recorded as a dependency, or unavailable even as an auditable relation.

The four statuses are derived from the operations available to a reasoner after projection. For a claim depending on a distinction d , the system must determine whether d is directly available for inference, recoverable by a bounded procedure, only recorded as a dependency, or unavailable even for audit. These cases license different answer forms. Directly available distinctions can support direct answers. Boundedly recoverable distinctions can support reconstruction or procedural answers. Merely recorded dependencies can support trace marking, routing, or qualification, but not recovery of the distinction itself. Dependencies that are neither recoverable nor traceable cannot support inference or audit from the current representation.

Thus the ladder is not intended as a complete ontology of epistemic states or a claim that all support relations are cleanly ordered. It is a minimal operational partition induced by answer-form admissibility: direct inference, bounded reconstruction, trace-only accountability, and unsupported or opaque dependency. More fine-grained statuses such as probabilistic reconstruction, partial reconstruction, conflicting provenance, or prior-dependent inference refine these cases rather than replace the need to distinguish among them.

Table 1: Dependency status and licensed answer forms.

Status	Support in representation	Licensed answer forms
Preserved	Distinction directly represented	Direct answer, ordinary correction
Reconstructible	Bounded recovery path available	Reconstruction, procedural answer
Traceable	Dependency recorded but not recoverable	Mark, qualify, route, audit
Opaque	Dependency unavailable or unauditable	Request expansion, route, defer, or refuse form

The status of a query-relevant distinction after projection can therefore be characterized by a base degradation ladder, often with qualifiers in practical systems.

A distinction is *preserved* when it remains directly represented in R in a form sufficient for the query. A distinction is *reconstructible* when it is not directly represented as an object-level feature, but can be recovered through an explicit bounded procedure using information available in R . A distinction is *traceable* when R records that a claim depends on a source, assumption, transformation, or lost relation, but does not contain enough information to recover the distinction itself. A distinction is *opaque* when the dependency is neither recoverable nor auditable from the representation.

The boundary between reconstructible and traceable depends on whether the representation

contains enough information to recover the distinction needed by the query, not merely enough information to identify where that distinction once came from. For example, a source hash, citation identifier, or provenance pointer may make a dependency traceable by recording that a claim depends on an external source. It is reconstructible only if the representation available to the reasoner also includes, retrieves, or otherwise supplies enough source content and recovery rules to determine the query-relevant distinction. If the pointer records that support exists but the relevant content is unavailable, the dependency is traceable rather than reconstructible. If even the dependency or source relation is not represented, the status is opaque.

A simple audit example illustrates the boundary between traceable and opaque loss. Suppose a decision system records that a recommendation depended on an external risk model, but does not preserve the model inputs, weights, or output explanation. The dependency is traceable: an auditor can see that the recommendation depended on the external model, even though the relevant distinction cannot be reconstructed from the current representation. By contrast, if the system preserves only the final recommendation and no record that an external model was used, then the dependency is opaque. The first case supports routing, audit, or qualification; the second supports neither reconstruction nor reliable accountability from the current representation.

This distinction is central for retrieval-augmented and tool-using AI systems. A retrieved document identifier, citation, tool-call trace, or audit log may make a dependency traceable without making the claim reconstructible. If the system cannot access the relevant source content, execution state, or recovery rule, then it should not treat the pointer as support for the claim. In safety and security settings, confusing traceability with reconstructability can produce false assurance: the system appears grounded because it has a pointer, but the representation available to the reasoner does not actually support the conclusion.

This ladder separates existence from operational support. Identifiability is an existence condition: if q is identifiable under P , then some function from R to the answer exists. Reconstructability is an operational condition: the system has an explicit recovery path, rule, derivation, proof, provenance structure, or bounded procedure that licenses the answer from the representation it actually uses. A query may be identifiable in principle while not being reconstructible for a particular system if the relevant recovery path is unavailable to that system.

Preserved and reconstructible distinctions can support inference. Traceable distinctions support accountability for transformation or loss, but not recovery of the original relation. Opaque loss supports neither inference nor audit. This ladder is therefore not a complete theory of provenance or explanation; it specifies what kind of support a downstream claim has after projection.

Dependency status is the invariant that later requirements are designed to preserve across reasoning steps, representation changes, correction attempts, and answer-form selection. Observability makes possible the detection or marking of unsupported dependency status. Bounded correction prevents a system from changing dependency status silently during repair. Non-collapse prevents claims supported at one level, such as a representation, intermediate state, or selected explanation, from being treated as if they were supported at another. The requirements that follow should therefore be read not as independent virtues, but as constraints on preserving dependency status under reasoning.

These categories are not intended to be exhaustive atomic states. They are base statuses that may require qualifiers in real systems. A dependency may be partially reconstructible, probabilistically reconstructible, reconstructible only under an explicit prior, supported by competing reconstruction paths, or traceable to conflicting provenance. In such cases, the relevant status should be reported with its qualifier rather than forced into a falsely crisp category. For example, a system may mark a claim as “probabilistically reconstructible under prior p ,” “traceable to source x but not recoverable from available content,” or “supported by two conflicting reconstruction

paths.” The ladder is therefore a discipline for preserving dependency status, not a claim that all support relations are cleanly ordered in practice.

This yields the practical test used throughout the paper: if a query is not identifiable under the current representation, then a single determinate answer requires either representation expansion, an explicit assumption or decision rule, or an answer form that marks the unresolved dependency.

5 Admissibility Requirements for Reasoning Under Projection

The requirements in this section are introduced as conceptual design constraints. Formal necessity, minimality, and sufficiency arguments are left for future work.

The formal result above does not by itself specify a complete reasoning policy. It specifies a dependency problem: after projection, a claim may be supported by distinctions that are preserved, reconstructible, merely traceable, or opaque. A structurally reliable reasoning process must therefore avoid treating these dependency statuses as interchangeable.

These requirements are privileged only in a limited sense: they correspond to three generic ways that dependency status can be misrepresented after projection. First, the status of a dependency may fail to become visible at all; this motivates observability of loss-relevant state. Second, the system may silently strengthen the status of a dependency during repair, treating traceable or opaque loss as if it had become reconstructible; this motivates bounded correction. Third, the system may transfer support across representational levels, treating support for a representation, proxy, intermediate state, or selected explanation as support for the underlying system or final claim; this motivates non-collapse of distinct levels.

Other mechanisms, such as calibration, provenance preservation, abstraction refinement, uncertainty representation, and compositional consistency, remain important. On this account, they are not rivals to the three requirements. They are implementation mechanisms, refinements, or domain-specific specializations. The three requirements are singled out because each blocks a distinct generic route by which unsupported dependency status can be promoted into apparently supported inference. The three requirements below should therefore be read as failure-preserving constraints rather than as a complete theory of reliable reasoning.

We describe the three requirements in turn.

5.1 Partial Observability of Loss-Relevant State

Observability of loss is necessarily partial and relative. A reasoning system need not be able to detect every distinction collapsed by projection; in general, it cannot. The requirement is not complete detection of loss, but preservation of detectable loss conditions relative to the representational language, metadata, or meta-representation available to the system. This includes:

- ambiguity between multiple consistent interpretations,
- absence of necessary variables or context,
- dependence on assumptions not encoded in the representation.

In the formal terms above, partial observability means that the system can sometimes detect evidence that the requested query q may not be constant over the equivalence class induced by P , relative to the distinctions it can express. When such evidence is available, the system must preserve it rather than treating the query as resolved.

Without observability, the system cannot distinguish between an identifiable query with a known answer and a non-identifiable query that only appears determinate after projection.

If such conditions are not observable within the system, they are effectively treated as resolved, even when they are not. This leads to outputs that appear complete but are based on unacknowledged gaps.

Observability does not require recovery of the missing information. It requires that detectable limits remain represented as limits. Failure of observability produces hidden loss: the system proceeds as if all relevant distinctions were present even when the representation no longer supports them.

5.2 Bounded Correction

Correction is bounded relative to a projection $P : S \rightarrow R$ when it can revise an incorrect result using only distinctions encoded in the representational state R . Here R includes not only object-level data, but any explicitly represented metadata, provenance information, uncertainty markers, priors, task constraints, or assumptions available to the reasoning process. If correction requires distinguishing between states that are equivalent under P and not distinguished anywhere in R , then the problem is not ordinary error correction within the representation. It requires representation expansion, new information, or an explicit assumption that changes the representational state.

The important point is not that correction must be small in a purely computational sense. It is bounded when the correction is licensed by distinctions already encoded in R , or by an explicit refinement that produces a new representation R' . A correction that silently imports distinctions not encoded in R is not correction within R ; it is an unmarked transition to a different representational state. For example, recomputing a formula from the same represented table is bounded correction; inventing a missing variable needed by the formula is not.

It is useful to distinguish two cases. In correction within R , the query is identifiable under the current representation and the system revises an incorrect answer using distinctions already available in R . In correction by refinement, the system explicitly moves from R to a richer or different representation R' by obtaining new information, adding a stated assumption, retrieving a source, or exposing metadata that was not previously represented. The second case may be admissible, but it is not correction within the original representation. It is bounded only if the transition from R to R' is explicit and the new information basis is marked. Thus bounded correction does not mean that a representation can never change. It means that correction must not silently change the information basis of the representation while presenting the result as if it followed from the original R .

Without bounded correction, a system may appear to repair an error while actually importing information not present in the representation, thereby simulating access to the original state space S .

This requirement separates valid correction from implicit reconstruction. If correcting a result requires unsupported new assumptions, wholesale replacement of the representation, or unbounded reconstruction of the problem state, then the issue is no longer ordinary error correction. It is either representational loss or a representation failure.

Bounded correction ensures that:

- errors can be repaired without silently changing the information basis of the representation,
- the distinction between error and loss is preserved in practice,

- correction remains a property of the reasoning process rather than an artifact of external intervention.

Failure of bounded correction leads to systems that appear to improve through repeated revision, but in fact introduce new assumptions or hidden state at each step.

5.3 Non-Collapse of Distinct Levels

A reasoning system must preserve role distinctions between objects that occupy different positions in the reasoning process. The relevant invariant is not that all levels have the same structure, but that objects with different roles must not be treated as interchangeable merely because they coincide under projection.

In particular, the system must not collapse:

- a representation with the underlying system it represents,
- an intermediate reasoning state with a final conclusion,
- a selected explanation with the full set of explanations consistent with the representation.

In each case, the collapse has the same form: a projected object is treated as if it had the role or authority of a richer object. A representation is treated as the world, a partial reasoning state as a conclusion, or one admissible model as the uniquely supported model. This makes projection loss invisible because the system can no longer track which distinctions were removed at which stage.

These distinctions may coincide in specific cases, but they are not equivalent in general. Collapsing them prematurely removes the ability to track where information originates and how conclusions are derived.

For example, treating a selected explanation as if it were uniquely determined by the data conflates the space of possible models with the subset supported by the representation. Similarly, reducing a multi-dimensional decision to a single value collapses distinct evaluative dimensions into a single outcome.

Failure of level separation results in premature convergence, where the system commits to a single interpretation without preserving alternative possibilities that remain consistent with the available information.

5.4 Structural Role of the Requirements

Together, these requirements preserve the distinction between what is known, what can be corrected, and what has been lost. A system that satisfies them does not eliminate failure; it keeps failures detectable and keeps reasoning grounded in the structure of the representation. When they are absent, outputs may appear coherent while the reasoning process has diverged from the information available to it.

6 Behavioral Consequences: Matching Answer Form to Dependency Status

Respecting these constraints does not make reasoning systems infallible. It changes the status of their failures. Unsupported claims become visible rather than hidden; ambiguity is represented rather than suppressed; correction remains tied to the information basis of the representation; and decisions under non-identifiability are marked as decisions rather than inferred conclusions.

These behavioral consequences motivate the operational uses described below. If the relevant failure mode is structural, then evaluation and interface design can target whether the required structure is preserved, reconstructed, traced, or opaque.

For AI reasoning systems, especially safety-critical ones, this changes what counts as a reliable answer. A reliable system should not merely be fluent, decisive, or consistent with a preferred output format. It should calibrate the form of its answer to the dependency status of the claim: direct when support is preserved, procedural when support is reconstructible, conditional when assumptions are required, trace-marked when support is external, and blocked or rerouted when the dependency is opaque.

7 Diagnostic Examples

These cases are not intended to estimate the frequency of projection-induced failures. They are diagnostic probes showing how the framework distinguishes unsupported determinacy, admissible conditional action, ordinary error, excessive abstention, and ambiguous dependency-status classification.

7.1 Case 1: Scalarization and Proxy Collapse

In this scenario, the system is required to produce an approval decision using a single numeric metric with no caveats. This constraint forces multiple dimensions of risk into a single value. The relevant query is not merely “which option maximizes the metric?” but “which decision is justified given reversibility, uncertainty, causal structure, and decision criteria?” If two underlying states map to the same scalar value while requiring different decisions along one of these dimensions, then the decision query is not identifiable under the scalar projection.

A structurally reliable response should identify the scalar projection as insufficient for the decision query. The result need not be refusal. The system may request a richer representation, branch over unresolved risk dimensions, or make an explicitly labeled policy decision under non-identifiability. The inadmissible move is to present the scalarized answer as an inference justified by the metric alone.

7.2 Case 2: Responsibility Assignment Under Causal Collapse

In this scenario, the system is asked to assign responsibility for a failure to a single individual, despite the presence of multiple interacting causes. The relevant query is not simply “who can be named as responsible?” but “which causal structure best explains the failure?” If distinct causal structures map to the same single-person attribution, then responsibility is not identifiable under that projection.

A structurally reliable response should avoid treating one selected actor as if that actor were uniquely determined by the representation. It may instead separate individual action, process failure, system-level interaction, and unresolved evidence. The point is not to avoid responsibility claims altogether, but to preserve the dependency status of the attribution being made.

7.3 Case 3: Traceability Without Source Reconstruction

A common language-model failure occurs when a system is asked to summarize or cite information that is not present in its context. The projected representation contains topic cues, stylistic pat-

terns, and plausible associations, but it does not contain the source-level distinction between what was actually observed and what is merely likely.

The relevant query is not simply “what answer is plausible?” but “which claims are supported by the provided source?” If two underlying states map to the same representation—one in which a claim is source-supported and one in which it is merely plausible—then source support is not identifiable under the projection. A structurally reliable response should separate supported claims from plausible but unsupported claims, request the missing source, or decline to assert source support.

7.4 Case 4: Decision Under Non-Identifiability

Not every non-identifiable query requires inaction. Suppose a triage system must choose between two actions under time pressure, but the representation does not contain enough information to determine which action is uniquely optimal. Two underlying states may map to the same representation while requiring different optimal actions under full information.

In this case, the query “which action is uniquely best?” is not identifiable under the projection. However, a decision may still be justified by an explicit rule, such as minimizing worst-case harm, deferring actions that would commit the system to loss beyond bounded correction, or selecting the option with a bounded fallback path. The decision is admissible only if it is marked as a policy choice under non-identifiability rather than as an inference uniquely determined by the representation.

7.5 Case 5: Ambiguous Dependency Status Under Limited Observability

Some cases cannot be cleanly classified from the current representation alone. Suppose a system gives an unsupported causal explanation, but the representation does not show whether the explanation came from hidden evidence, a valid prior, or an invented assumption. From the output alone, the failure may look like representational loss, ordinary error, or unjustified decision under uncertainty.

In this case, the framework does not by itself determine the correct classification. It identifies what must be made observable: whether the explanation is supported by information in the representation, by an explicit prior or decision rule, or by an unsupported assumption. This limit case is important because the framework does not eliminate projection; it specifies what additional structure must be exposed for classification to be reliable.

7.6 Summary of Behavioral Modes

The examples distinguish restructuring, conditional decision, representation expansion, trace marking, and refusal of the requested answer form as possible responses to non-identifiability. They also show that the framework should not be used as a blanket abstention policy: when a query is identifiable under the current representation, ordinary correction remains appropriate. The central requirement is not caution for its own sake, but matching the answer form to the dependency status of the claim.

8 Applications

Dependency status can be used as a design and evaluation lens for several AI reasoning, safety, and security problems.

Security copilots, code assistants, and incident response. Security and incident-response assistants often reason from compressed code context, alert summaries, deployment traces, and partial telemetry. The framework requires such systems to distinguish vulnerabilities or root causes supported by represented evidence from recommendations that depend on missing code paths, runtime configuration, threat models, event ordering, or logging conditions. When these distinctions are absent or only traceable, the system should mark the missing dependency, propose discriminating tests, request representation expansion, or offer bounded mitigations rather than present a single unsupported cause or fix as determined.

Retrieval-augmented generation and source grounding. RAG systems frequently expose provenance pointers, citations, or retrieved snippets. The dependency-status ladder distinguishes traceability from reconstructability: a pointer records a dependency, but only available source content and recovery rules can support the claim. This provides a concrete audit target for hallucination and false-grounding failures.

Agentic systems. Agents act from compressed internal state: task instructions, memory, tool outputs, environmental observations, and inferred user intent. Before acting, an agent should determine whether the distinctions required for action are preserved or reconstructible: authorization, reversibility, environmental state, user intent, and safety constraints. When these distinctions are opaque, the agent should request representation expansion, route to a human or tool, or act only under an explicit bounded policy.

User modeling and Theory-of-Mind-like behavior. Dependency status also applies to AI systems that model users or other agents. A system often infers what a user knows, wants, intends, believes, or cannot distinguish from partial interaction traces. These inferences are projection-sensitive: observed behavior may preserve some distinctions while leaving others merely plausible, traceable to prior context, or opaque. A safety-relevant user model should therefore mark whether claims about another agent’s state are directly observed, reconstructible from interaction, dependent on an explicit assumption, or unsupported. This does not require treating the AI system as having humanlike inner experience. It treats Theory-of-Mind-like behavior as reasoning under projection over another agent’s representation.

Evaluation and benchmark design. Standard evaluation may reward correct-looking answers even when the representation does not support them. Projection-aware evaluation should test whether systems change behavior when observability changes: refusing unsupported determinacy at low observability, ranking hypotheses under partial evidence, and becoming appropriately decisive when the relevant distinction becomes observable. This makes dependency status a measurable target rather than a purely philosophical concern.

9 Why Existing Methods Can Miss This Failure

Existing methods can improve performance within a representation without testing whether the representation preserves the distinctions required by the query. Optimization can improve a proxy while hiding loss of task-relevant structure. Prompting can make reasoning more detailed without verifying that the needed dependency is represented. Evaluation can reward a plausible answer while failing to test whether a determinate answer was identifiable from the available evidence.

The common mismatch is that these methods often operate over R , while the failure arises from the projection $P : S \rightarrow R$. A source-grounding benchmark may check whether an answer contains a citation, but not whether the cited content preserves the distinction needed by the claim. An incident-response benchmark may reward naming a plausible cause, while failing to penalize the system for ignoring that telemetry cannot distinguish among several causes.

This does not mean optimization, prompting, or evaluation are misguided. It means they require an additional structural check: whether the claim’s dependency status is preserved, reconstructible, traceable, or opaque under the representation being used.

10 Relation to Existing Frameworks

The argument in this paper overlaps with several established lines of work. In statistics and causal inference, identifiability asks whether a target quantity can be determined from the available observational structure [8]. In statistical decision theory, comparison of experiments and sufficiency ask when one signal or observation preserves enough information for a class of decisions [2]. In partially observable decision processes, agents must act over incomplete state representations [6]. In work on state abstraction, compressed states are evaluated by whether they preserve value, policy, reward, transition, or model-relevant properties [7]. In formal methods and abstract interpretation, abstraction maps concrete states into reduced domains while preserving selected properties [4]; counterexample-guided abstraction refinement studies how an abstraction may be refined when it is too coarse for the property being checked [3]. In information-theoretic approaches such as the information bottleneck, compressed representations are evaluated by what task-relevant information they retain [11].

This distinction is especially important for partial-observation models. Partial-observation methods, including POMDP-style accounts, can represent uncertainty over hidden states and help identify ambiguity, underspecification, or missing evidence. But hidden state is not the same as projected-away structure. A distinction may be unavailable from the current representation rather than merely unknown within it. Sensing hiddenness can indicate that the representation is incomplete; reconstructibility governance determines whether the claim remains admissible, must be weakened, requires representation expansion, can be traced, or should be blocked.

The present paper does not claim that these frameworks are equivalent, nor does it replace their domain-specific accounts of identifiability, abstraction, observability, or belief-state update. Instead, it identifies a shared reasoning-system failure pattern: a downstream reasoner is asked to draw a determinate conclusion from a representation that may not preserve the distinctions required by the query. Prior work often specifies when a property is identifiable, when an abstraction is sound, when a statistic is sufficient, or when a belief state supports action. The failure analyzed here occurs when a reasoning system behaves as though the relevant identifiability, sufficiency, or preservation condition holds when it does not. In particular, partial-observation methods are useful for sensing hiddenness and maintaining uncertainty over represented possibilities, but they do not by themselves establish that a query-relevant distinction survived projection or is boundedly reconstructible from the current representation.

The distinct contribution of the present paper is therefore behavioral and operational. It treats non-identifiability under projection not only as a formal limitation, but as a source of answer-form misclassification: the system presents a claim as if its dependency status were preserved, reconstructible, or grounded when the current representation supports only traceability, qualification, routing, or refusal of the requested form. The paper’s focus is therefore not only whether a property is identifiable in a formal model, but how a reasoning system should behave when the representation

available to it does not support the query it is asked to answer. In particular, the framework emphasizes the behavioral misclassification that occurs when a system treats a non-identifiable query as if it were an ordinary recoverable error.

The error–loss distinction is therefore a diagnostic constraint on reasoning behavior. If the query is identifiable under the current representation and the system answers incorrectly, the failure is ordinary error. If the query is not identifiable, then additional optimization, prompting, explanation, or evaluation within the same representation cannot by itself recover the missing distinction. The system must instead expand the representation, obtain new information, introduce an explicit assumption or decision rule, weaken the claim, or mark the dependency as traceable or opaque.

11 Operational Implications

A minimal implementation can treat dependency-status classification as a four-step diagnostic rather than as an oracle:

1. Identify the claim or action being licensed.
2. Identify the distinction on which that claim depends.
3. Check whether the distinction is preserved, reconstructible by a bounded procedure, merely traceable, or opaque in the operative representation.
4. Select an answer form licensed by that status: direct answer, bounded reconstruction, conditional answer, branch over assumptions, request for representation expansion, trace marking, routing, deferral, or refusal of the requested form.

The purpose of this section is not to define a complete protocol, evaluation framework, or control layer. It is to show how the dependency-status ladder naturally yields operational targets: answer-form governance, evaluation under projection stress, interface-level representation repair, and audit fields for downstream systems.

This procedure may itself be uncertain. In that case, the system should mark uncertainty about dependency status rather than silently promote the claim to a stronger status.

11.1 Evaluation by Structural Shape

Evaluation can be organized around whether a response preserves the structural shape required by the query, rather than only whether it matches a domain-specific target answer. A responsibility query can be evaluated for preservation of individual, process, and system levels; a high-stakes decision for visibility of irreversible tradeoffs and residual risk; a source-grounded answer for separation between observed support and plausible reconstruction; and a compositional policy problem for revalidation under interaction or scale. Such diagnostics ask whether the answer has the right admissibility structure: what distinction is needed, whether it is preserved or reconstructible, whether loss is visible, and whether the system has confused inference with decision. An evaluation harness could operationalize these targets by running controlled question sets under different prompt and scaffold conditions.

11.2 Reasoning Scaffolds and Constraint Activation

The same dependency-status distinctions can be used to constrain generation. A reasoning scaffold can require the system to preserve observability of loss-relevant state, avoid collapse of conceptual levels, keep correction bounded, distinguish error from loss, and revalidate conclusions under scale or composition. Such scaffolds do not prescribe a single reasoning style. They constrain which transformations are admissible when the system moves from a projected representation to an answer.

In addition, different tasks may require attention to different dependency-status risks. A source-grounding task may foreground observability and provenance; a responsibility-assignment task may foreground non-collapse of levels; a deployment decision may foreground bounded correction and scale revalidation. Task-specific activation of these dependency-status risks provides a control layer over reasoning without changing the underlying structural requirements.

11.3 Interface and Question-Formation Uses

Dependency-status distinctions can also be used before answer generation, as interaction lenses for forming better questions. A user interface need not treat the query as fixed. It can help users select objects, group them, specify relations, and choose a reasoning lens such as observability, reconstructability, traceability, admissibility, scale, or non-collapse. The resulting query is then not merely a request for an answer, but an explicit statement of which distinctions must be preserved during reasoning.

On this view, question formation is itself a projection-management task. The user and system jointly determine which distinctions matter before inference proceeds. This is especially important in settings where the initial question collapses roles, scales, sources, or decision criteria that must remain separate for the answer to be admissible.

In interactive systems, clarifying questions are one mechanism for representation repair. A clarification request is not merely conversational hesitation or a user-interface convenience. It is a way of enriching, refining, or changing the projection so that the requested query becomes identifiable, boundedly reconstructible, or explicitly conditional. Conversely, if the missing distinction does not affect the dependency status of the requested claim, the system should not treat clarification as required. The point is not to ask more questions, but to ask when the current representation does not support the requested answer form.

11.4 Audit and Dependency Status

Finally, the dependency-status ladder suggests audit fields for downstream systems. A claim can be marked according to whether its required dependency is preserved, reconstructible, traceable, or opaque. Such markings do not guarantee correctness, but they make visible whether a conclusion is supported by the representation, recovered through an explicit path, dependent on a recorded but unrecoverable relation, or unsupported by observable provenance.

This allows downstream users or systems to distinguish claims that are inferentially supported from claims that are merely decisions, assumptions, or unresolved dependencies under non-identifiability.

Taken together, these uses show how dependency status can guide downstream scaffolds, evaluation harnesses, runtime systems, interfaces, and audit mechanisms without requiring a single domain-specific answer metric.

12 Scope and Limits

This paper describes structural conditions for reasoning under projection. It does not attempt to provide a complete theory of reasoning, nor does it claim to resolve all sources of uncertainty or failure. This section clarifies the scope of the claims and the limits of what these constraints guarantee.

12.1 What This Does Not Solve

Respecting the constraints described in this paper does not eliminate the fundamental challenges associated with reasoning over incomplete or compressed representations.

In particular, it does not:

- guarantee that a system will produce correct conclusions,
- eliminate ambiguity in cases where multiple interpretations remain consistent,
- recover distinctions that are not preserved or boundedly recoverable under the current representation,
- ensure that all relevant variables are present in the representation,
- prevent all forms of bias or misalignment in decision-making.

These limitations are not defects of the account. They follow directly from the constraints imposed by projection. Any system operating over a reduced representation must accept that some distinctions cannot be made and some questions cannot be resolved.

12.2 Dependence on Representation

The behavior of a reasoning system under this framework depends on the representation over which it operates. Different representations preserve different distinctions, and therefore introduce different forms of loss.

Improving reasoning performance may require modifying the representation itself, not just the reasoning process applied to it. However, such changes alter the projection and therefore change the conditions under which reasoning operates.

This paper does not prescribe specific representations. It provides constraints that apply once a representation has been chosen.

12.3 Mechanism and Evaluation as Downstream Work

The present paper specifies the structural failure mode rather than a complete implementation. A detector for representational loss would need to identify when a query depends on distinctions that are neither preserved, nor boundedly reconstructible, nor explicitly assumed in the current representation. A dependency-status implementation would need to represent whether a claim is preserved, reconstructible, traceable, or opaque.

Evaluation should therefore test not only whether a system reaches a target answer, but whether it changes behavior appropriately as the representation changes. One natural benchmark pattern is to compare compressed and enriched versions of the same task: when the compressed representation does not preserve the distinction required by the query, the system should avoid unsupported determinacy; when the enriched representation restores the distinction, the system should become

correspondingly more decisive. This tests whether the system distinguishes representational loss from ordinary error rather than merely becoming more cautious.

12.4 Limits of Observability

While the error-loss distinction requires observability of loss-relevant state, this observability is itself subject to projection. A system may be unable to represent certain forms of loss because the representation does not include the necessary structure.

This creates an important limitation: loss can only be detected relative to distinctions that the representation is capable of expressing. A system cannot observe every loss introduced by its own projection; it can only mark the boundaries where the current representation no longer supports a requested query.

In such cases, the system may be unable to indicate the specific lost distinction, even if loss exists. It may still be able to mark a weaker diagnostic condition, such as missing variables, unsupported assumptions, or insufficient provenance. This places a practical limit on how completely loss can be made visible within any given representation.

The practical target is therefore improved loss visibility, not complete loss detection.

13 Conclusion

AI systems reason from compressed representations: prompts, summaries, retrieved snippets, tool outputs, logs, metrics, memory, and policy artifacts. Compression is unavoidable, but it can collapse distinctions that later claims require. When a query is not identifiable under the current representation, additional reasoning over that representation cannot recover the missing distinction.

The central error analyzed in this paper is misclassifying representational loss as recoverable inference error. This produces fluent but unsupported determinacy: hallucinated source support, overconfident root-cause attribution, unsafe action under missing state, or metric success that hides loss of relevant structure.

The proposed dependency-status ladder—preserved, reconstructible, traceable, and opaque—does not guarantee correctness. It constrains how claims relate to the representations that support them. A reliable system should answer directly when support is preserved, reconstruct through bounded procedures when available, mark traceable dependencies when recovery is absent, and avoid presenting opaque dependencies as determined. In invariant terms, the ladder tracks whether the dependency structure required by a claim has survived projection, can be restored by bounded reconstruction, remains only traceable, or has become unavailable to the reasoning process.

Reasoning under projection is therefore not an edge case. It is the ordinary condition of AI systems operating over bounded context, retrieval, tools, summaries, and metrics. The admissibility question is whether such systems can preserve the difference between what their representations determine, what they can reconstruct, what they can merely trace, and what they have lost.

Future work can develop this entry-point claim into formal minimal conditions, geometric dynamics of projection-induced drift, response-form governance protocols, empirical evaluation methods, and adaptive control mechanisms.

References

- [1] Dario Amodei et al. Concrete Problems in AI Safety. arXiv:1606.06565, 2016.

- [2] David Blackwell. Equivalent comparisons of experiments. *The Annals of Mathematical Statistics*, 1953.
- [3] Edmund M. Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement. *CAV*, 2000.
- [4] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. *POPL*, 1977.
- [5] Charles Goodhart. *Problems of Monetary Management: The U.K. Experience*. 1975.
- [6] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 1998.
- [7] Lihong Li, Thomas J. Walsh, and Michael L. Littman. Towards a unified theory of state abstraction for MDPs. *ISAIM*, 2006.
- [8] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2009.
- [9] Leonard J. Savage. *The Foundations of Statistics*. 1954.
- [10] Claude Shannon. A Mathematical Theory of Communication. *Bell System Technical Journal*, 1948.
- [11] Naftali Tishby, Fernando C. Pereira, and William Bialek. The information bottleneck method. 1999.